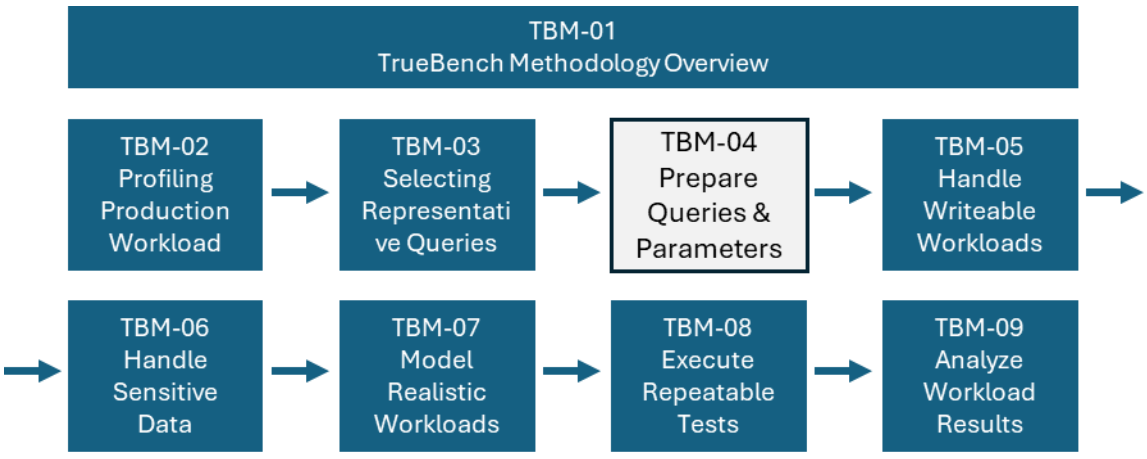
Prepare Queries & Parameters

Reducing Millions of Queries to a Representative Test Workload.

Author: Douglas H Ebel

Data Warehouse Architect
Business Value and Performance Assessment

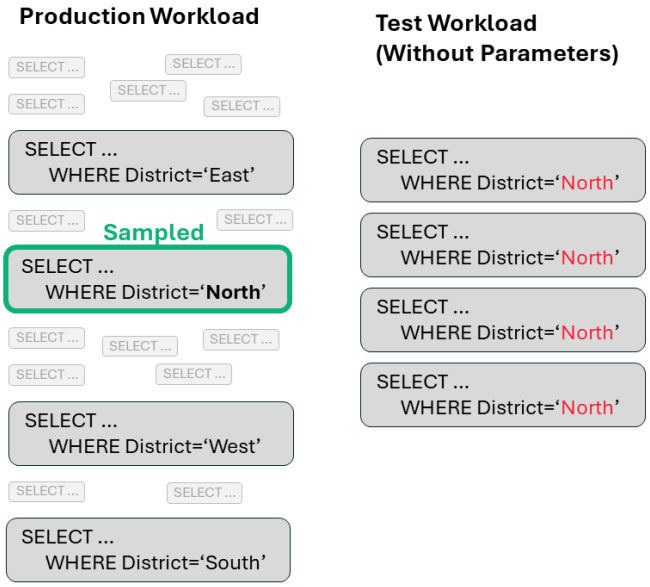
Creator of TdBench and TrueBench



Introduction

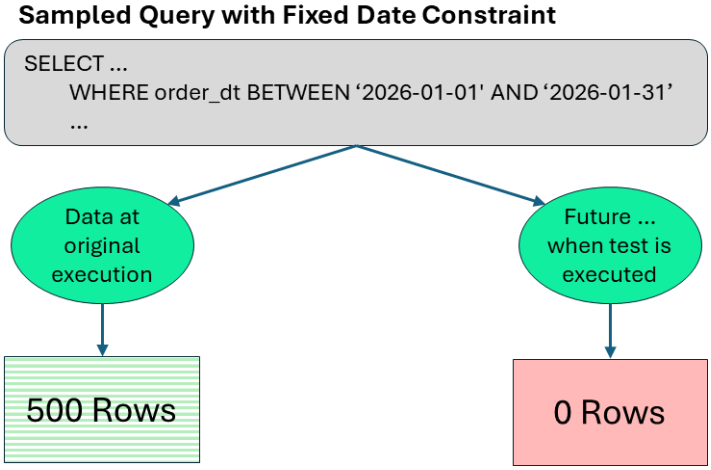
The queries selected in TBM-03 represent the structure of a production workload, but they are not yet suitable for execution as a realistic test. Two issues must be addressed:

First, each query typically contains fixed literal values. In production, those same logical queries are executed by many users across different customers, products, regions, and time periods. A single set of literals does not represent this diversity of usage.



A single representative query does not create a representative workload

Second, literal values often refer to specific time periods or business entities that may no longer be relevant. When executed against current data, such queries may return no rows or produce results that do not reflect current activity.



TBM-04 addresses these issues by converting each selected query into a parameterized form. Literal values are replaced with parameter markers (:1, :2, ...), and parameter values are generated at execution time to reflect current data and realistic usage patterns.

This conversion step requires identifying opportunities for parameterization and generating parameter values that preserve valid and meaningful data access.

In TBM-03, several hundred work units were selected from thousands of queries to make this effort manageable. Even with that reduction, the conversion remains a non-trivial task.

To support this step, TrueBench provides **TrueBench Builder** as a workflow productivity aid. It assists:

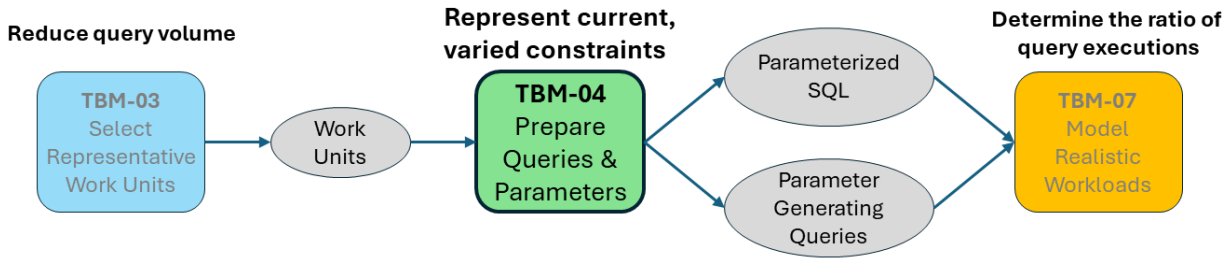
- identifying candidate constraints within each query
- enabling selective parameterization under analyst control
- generating initial parameterized SQL and parameter-generation queries
- supporting iterative refinement where automation is not sufficient

The process remains analyst-driven. The tool accelerates the work and enforces safe patterns, but the analyst determines what constitutes a valid and realistic representation of the workload.

The result of TBM-04 is a set of:

- parameterized SQL statements
- parameter-generation queries

These artifacts form the executable workload used by TrueBench in subsequent testing.



TBM-04 Process Overview

Work units identified in TBM-03 may represent either:

- single query, or
- group of related queries executed together to produce a report or dashboard,

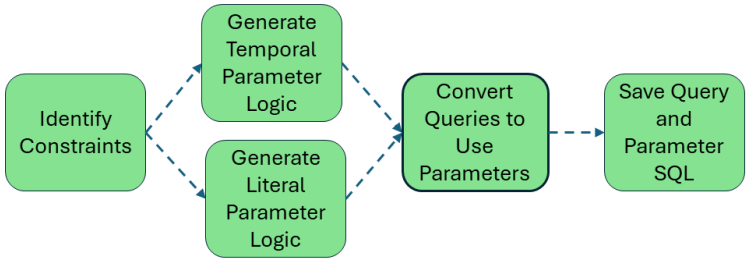
often involving intermediate steps or temporary results.

TBM-04 begins with the simpler case **single-query work units** and extends the same approach to **multi-step work units** once the parameterization patterns are established.

The conversion process follows a consistent sequence:

- Identify constraints in each query
- Generate parameter values
  - temporal values based on execution time
  - literal values sampled from data
- Convert the query to use parameter markers
- Save the parameterized query and its parameter-generating query

This sequence transforms each work unit into an executable form that can be used in realistic workload testing.



Identify Constraints

Each query contains constraints expressed as literal values. These define the specific slice of data accessed by the query and must be converted into parameters to make the workload executable and repeatable.

Constraints typically appear in the following forms:

- **Comparison predicates**
  - =, >, >=, <, <=

  Example:
  - c_mktsegment = 'HOUSEHOLD'
  - l_quantity < 25

- **Range predicates**
  - BETWEEN ... AND ...

  Example:
  - l_discount BETWEEN 0.06 AND 0.08

- **Set predicates**

    - IN (...)

  Example:
    - c_region IN ('NORTH', 'SOUTH')

- **Temporal** predicates

  Example:
    - l_shipdate >= DATE '2026-01-01'

In addition to identifying the predicate type, it is important to determine:

- The table or alias associated with each constrained column
- Whether constraints reference a single table or multiple tables

Constraints that reference a single table can typically be parameterized using values sampled from that table.

Constraints that reference multiple tables require a parameter-generating query that reproduces the necessary joins to maintain valid combinations of values.

Set predicates using IN lists or subqueries may also require more advanced parameter generation to avoid producing invalid or non-existent combinations.

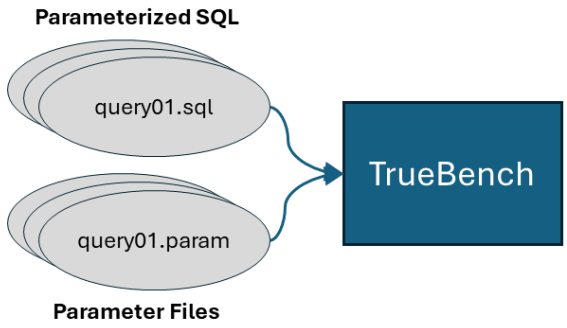
The goal is not to capture every possible constraint, but to identify a set of constraints that can produce valid and representative query execution when used with parameters.

> **Hint:**
> TrueBench Builder scans each query to identify candidate constraint expressions and their associated tables.

Generate Parameter Values

For each parameterized query, a corresponding parameter-generating query is created.

These parameter-generating queries are executed prior to each test run to produce a valid set of parameter values based on the current data. The generated values are written to a file that shares the same root name as the associated parameterized query.



One parameter file is generated for each parameterized query

TrueBench automatically associates each query with its parameter file during execution.

In some cases, multiple queries may use the same parameter set. When parameter generation is time-consuming, the generated values may be reused. If duplicated, it is recommended to shuffle the rows to reduce the likelihood of concurrent queries accessing the same data at the same time.

> **Hint:**
> TrueBench Builder assists in defining parameter-generation logic based on selected constraints and data sources.

Generate Parameter Values — Temporal Constraints

If the selected constraints are entirely temporal, the parameter-generating query needs to produce only a single row. If the selected constraints are a mix of Temporal and literal, then the temporal parameter values will be the same on each generated row.

The generated values should preserve the same relationship to the time of execution as existed in the original query.

For example, a query executed at:

2026-02-04 09:05:21

may contain a constraint referencing:

DATE '2026-01-01'

This represents the first day of the prior month relative to the execution time. When the test is run later, the parameter-generating logic should reproduce that same relationship using the current date.

Example:

```
SELECT
  ADD_MONTHS(TRUNC(CURRENT_DATE, 'MONTH'), -1) AS start_of_prior_month
```

This produces a single parameter value aligned to the current execution time, ensuring the query continues to reference data from the prior month.

> **Hint:**
> TrueBench Builder detects temporal patterns and generates expressions relative to the current execution time..

Generate Parameter Values — Literal Constraints

For literal constraints, parameter values are generated by sampling valid data from the underlying tables.

When all selected constraints reference a single table, parameter generation is straightforward. A sample of rows (for example, 100) can be selected from that table, returning the constrained columns in the desired order.

When constraints reference multiple tables, the parameter-generating query must ensure that the combinations of values are valid. This typically requires reproducing the relevant join logic from the original query. Depending on data cardinality, a `GROUP BY` may be used to return distinct combinations of values.

Temporal values can be included in the same parameter-generating query by adding expressions to the `SELECT` list. These values will be constant across all rows and combined with the sampled literal values.

The order of columns in the `SELECT` list determines how values are assigned to parameter markers (`:1`, `:2`, ...). For clarity, it is useful to document the parameter positions with comments in the parameter-generating query.

If a parameter file contains more columns than are referenced by a query, the additional values are ignored. This allows a single parameter set to be reused across multiple queries when appropriate.

> **Hint:**
> TrueBench Builder generates initial parameter queries for single-table constraints and highlights cases requiring manual extension.

When writing parameter files, choose a delimiter that will not appear in the data values (for example, avoid commas if values may contain commas).

Convert Literals to Parameters

After the parameter values to be generated have been defined, the original SQL can be transformed into its parameterized form.

For each constraint selected for parameterization, it is helpful to preserve the original expression as a comment and then place the revised expression directly below it. This makes the transformation easier to review and simplifies later debugging. Examples:

> **Hint:**
> TrueBench Builder produces an initial parameterized version of the query while preserving original constraints for reference.

Original constraints:

```
l_shipdate >= date '1993-01-01'
   and l_shipdate < date '1993-01-01' + interval '1' year
and c_mktsegment = 'HOUSEHOLD';
```

Revised constraints for parameterized query:

```
-- l_shipdate >= date '1993-01-01'
--   and l_shipdate < date '1993-01-01' + interval '1' year
-- and c_mktsegment = 'HOUSEHOLD';

l_shipdate >= date '%1'
   and l_shipdate < date '%1' + interval '1' year
and c_mktsegment = '%2';
```

Save the Parameterized Query and Its Parameter File

Each parameterized query should be saved with a filename that uniquely identifies it within the workload. The associated parameter file should use the same root name.

Example:

- `sales_001.sql`
- `sales_001.param`

Using a common root name allows TrueBench to associate the query with its parameter file automatically.

It is generally best to save parameterized queries in one or more query directories and save parameter files in a separate parameter directory. This keeps the executable SQL separate from the generated data values while preserving their association through the shared file name.

If queries are likely to be reused in different workloads by business unit or subject area, the queries may be separated as they are saved. For example:

- `sales/sales_001.sql`
- `mkt/mkt_001.sql`
- `fin/fin_001.sql`

Initial serial testing may later suggest moving queries into different directories to simplify TrueBench queue construction. Parameter files, however, may remain in a single directory as long as the root query name is unique across the workload.

A simple naming convention is recommended:

- use a short business or workload prefix
- follow with a sequence number that is zero-padded on the left
- keep the same root name for the query and parameter file

Examples:

- `sales_001.sql` and `sales_001.param`
- `fin_014.sql` and `fin_014.param`
- `mkt_027.sql` and `mkt_027.param`

Extend to Multi-Step Work Units

For work units consisting of multiple related queries:

- the same parameter set must remain consistent across all steps
- parameters are generated once and reused across the sequence

This preserves the logical relationships between queries that operate together to produce a report or dashboard.

Outcome

The result of TBM-04 is a set of executable artifacts:

- parameterized SQL statements
- parameter-generating queries

These artifacts enable repeatable execution of realistic workloads against current data, preserving both query structure and variability of usage.

This transformation allows subsequent testing to measure how the system behaves under conditions that reflect actual usage, rather than fixed or outdated query inputs.

Future updates will include additional guidance on using TrueBench Builder, including installation steps and a companion demonstration.